



Guia do Aluno — Clean Code e Refatoração

UC: Programação Back-End | **Bloco 2 — Aula de 02/09/2026**

Projeto: API de Gestão da Livraria (SA1)

O que você vai conseguir fazer ao final

- ☐ Explicar o que é refatoração e por que ela é uma prática contínua, não um evento único
- ☐ Extrair uma constante nomeada de um número mágico
- ☐ Quebrar um método que faz coisa demais em métodos menores
- ☐ Aplicar um checklist de clean code nas próprias classes
- ☐ Revisar o código de um colega de forma construtiva

Onde trabalhar: hoje é no seu repositório pessoal (`programacao-backend`), nas classes que você já criou.

1. O que é refatorar

REFATORAR = mudar a FORMA do código sem mudar o COMPORTAMENTO dele.

Antes de refatorar: o programa faz X.

Depois de refatorar: o programa continua fazendo X.

Só o código por dentro ficou mais fácil de ler e de mudar depois.

Regra de ouro: toda refatoração termina com um teste de que nada mudou. Rode o programa antes, anote a saída, refatore, rode de novo, compare. Se a saída for igual, a refatoração foi segura.

💡 Lembra do número `2.5` em `calcularFrete()` , que apareceu como exemplo em 12/08? Ele nunca foi corrigido de verdade — ficou como diagnóstico. Hoje ele vira o primeiro conserto real do dia.

2. Refatoração é hábito de equipe, não faxina de última hora

Quando o código é só seu, ninguém mais lê. Quando é de um grupo de 4 pessoas trabalhando na mesma API, código confuso vira um imposto que todo mundo paga — inclusive você, daqui a duas semanas, quando não lembrar mais por que escreveu aquilo daquele jeito.

Times de desenvolvimento profissional refatoram o tempo todo, sempre que voltam a mexer em alguma parte do código. E revisam o código uns dos outros o tempo todo também — não porque alguém programa mal, mas porque um segundo par de olhos sempre encontra algo que o primeiro não viu. Isso se chama **code review**, e é prática normal, não acusação.

3. Refactor guiado 1 — a constante mágica

Abra seu `LivroFisico.js`. Deve ter algo parecido com isto:

```
calcularFrete() {  
  return this.#peso * 2.5;  
}
```

Passo 1 — teste antes. Rode e anote o resultado do frete para um livro de teste (ex.: 0.6kg deve dar R\$ 1,50).

Passo 2 — extraia a constante, no topo do arquivo, fora da classe:

```

const Livro = require("./Livro");

const PRECO_POR_KG = 2.5;

class LivroFisico extends Livro {
  #peso;

  constructor(titulo, autor, preco, estoque, peso) {
    super(titulo, autor, preco, estoque);
    this.#peso = peso;
  }

  get peso() {
    return this.#peso;
  }

  calcularFrete() {
    return this.#peso * PRECO_POR_KG;
  }

  descrever() {
    super.descrever();
    console.log("Tipo:    Fisico");
    console.log("Peso:    " + this.#peso + "kg");
    console.log("Frete:   R$ " + this.calcularFrete().toFixed(2));
  }
}

module.exports = LivroFisico;

```

Passo 3 — teste depois. Rode de novo. O resultado precisa ser idêntico ao do Passo 1.

O ganho: se a Livraria decidir cobrar R3,00 *por kg em vez de* R 2,50, existe um lugar só para mudar, e o nome `PRECO_POR_KG` já explica o que aquele número significa.

Se você não tem `LivroFisico.js` ainda, aplique o mesmo raciocínio em qualquer número solto que encontrar em outra classe sua.

4. Refactor guiado 2 — um método que faz coisa demais

Veja este exemplo, com um método `resumo()` fazendo três coisas ao mesmo tempo — listar os itens, calcular o total e decidir sobre frete grátis:

```
resumo() {  
  console.log("--- Itens do carrinho ---");  
  this.#livros.forEach((livro) => {  
    console.log(livro.titulo + " - R$ " + livro.preco);  
  });  
  let total = 0;  
  this.#livros.forEach((livro) => { total = total + livro.preco; });  
  console.log("Total: R$ " + total.toFixed(2));  
  if (total > 100) {  
    console.log("Voce ganhou frete gratis!");  
  } else {  
    console.log("Faltam R$ " + (100 - total).toFixed(2) + " para frete gratis.");  
  }  
}
```

Depois de quebrado em métodos menores, cada um com uma responsabilidade só:

```
const VALOR_MINIMO_FRETE_GRATIS = 100;

class Carrinho {
  #livros;

  constructor() {
    this.#livros = [];
  }

  adicionarLivro(livro) {
    this.#livros.push(livro);
  }

  listar() {
    console.log("--- Itens do carrinho ---");
    this.#livros.forEach((livro) => {
      console.log(livro.titulo + " - R$ " + livro.preco);
    });
  }

  calcularTotal() {
    let total = 0;
    this.#livros.forEach((livro) => {
      total = total + livro.preco;
    });
    return total;
  }

  mostrarStatusFrete(total) {
    if (total > VALOR_MINIMO_FRETE_GRATIS) {
      console.log("Voce ganhou frete gratis!");
    } else {
      const faltam = VALOR_MINIMO_FRETE_GRATIS - total;
      console.log("Faltam R$ " + faltam.toFixed(2) + " para frete gratis.");
    }
  }

  resumo() {
    this.listar();
    const total = this.calcularTotal();
    console.log("Total: R$ " + total.toFixed(2));
    this.mostrarStatusFrete(total);
  }
}
```

```
}  
  
module.exports = Carrinho;
```

Repare: `resumo()` agora só **chama** os outros métodos, sem fazer nada sozinho. A saída na tela é exatamente a mesma de antes — só o código por dentro ficou mais fácil de ler, de testar e de reaproveitar.

Se você tiver um método parecido em qualquer classe própria fazendo mais de uma coisa, aplique a mesma quebra. Se não tiver, use este exemplo do `Carrinho` como treino.

5. O checklist de clean code

Use este checklist para revisar suas próprias classes hoje, e depois para revisar o código do colega.

#	Item	O que verificar
1	Nomes claros	Classes, métodos e variáveis têm nomes que dizem o que são, sem abreviações confusas
2	Sem números mágicos	Todo valor com significado (preço, limite, taxa) tem uma constante nomeada
3	Métodos pequenos	Cada método cabe na tela sem rolar e faz uma coisa só
4	Sem duplicação	Código repetido em métodos ou classes parecidas foi reaproveitado, não copiado
5	Dados protegidos	Atributos com regra de negócio usam <code>#</code> , <code>get</code> e <code>set</code> , não ficam expostos livremente
6	Um arquivo, uma classe	Cada arquivo exporta uma única classe, com <code>module.exports</code> no final
7	Comentários úteis	Se existem comentários, eles explicam o "porquê", não repetem o que o código já diz

6. Code review em duplas

Hoje, dentro do seu grupo de projeto, formem duplas. Cada um mostra o próprio código para o outro, usando o checklist acima como roteiro da conversa.

Como dar uma boa revisão:

- Seja específico. Em vez de "está bom" ou "está ruim", aponte a linha e o item do checklist.
- Pergunte antes de afirmar. "Por que esse número está direto aqui?" abre conversa melhor que "isso está errado".
- Lembre que o objetivo é achar pontos de melhoria, não aprovar ou reprovar o colega. Toda revisão de código de verdade parte do princípio de que sempre existe algo a ajustar.

Como receber uma revisão:

- Ouça antes de se defender. Nem toda sugestão precisa virar mudança, mas toda sugestão merece ser entendida primeiro.
- Se aceitar a sugestão, aplique o ajuste na hora, com o colega por perto para confirmar.



Checklist

- ☐ Constante `PREC0_P0R_KG` extraída, testada antes e depois
 - ☐ Ao menos um método longo quebrado em métodos menores
 - ☐ Checklist de clean code aplicado nas próprias classes
 - ☐ Participei da revisão do código de um colega
 - ☐ Recebi a revisão do meu próprio código
 - ☐ `git push` feito
-

Cola rápida

```
// Constante nomeada em vez de número mágico
const PRECO_POR_KG = 2.5;

// Método longo dividido em métodos menores
metodoGrande() {
  this.parteA();
  this.parteB();
  this.parteC();
}
```

Item do checklist	Pergunta para si mesmo
Nomes claros	Dá para entender sem abrir o resto do código?
Sem números mágicos	Esse número tem um nome que explica o que ele significa?
Métodos pequenos	Cabe na tela sem rolar?
Sem duplicação	Esse trecho já existe em outro lugar?
Dados protegidos	Esse atributo tem regra de negócio que precisa de <code>#</code> ?
Um arquivo, uma classe	Tem <code>module.exports</code> no final?
Comentários úteis	O comentário explica o porquê, ou só repete o código?